

SHRSS – AEM Assets API Comparison Guide

1	Overview	2
1	Quick decision guide	2
2	Shared prerequisites (for both approaches)	3
3	Option A – Classic Assets HTTP API + Direct Binary Upload	4
3.1	When to use this option	4
3.2	Technical account & access token (classic)	4
3.3	Folder & asset CRUD via Assets HTTP API	5
3.3.1	List folder contents (Read).....	5
3.3.2	Create a folder under photography (Create – folder)	5
3.3.3	Create a new asset via Direct Binary Upload (Create – asset)	5
3.3.4	Update asset metadata & binary (Update)	7
3.3.5	Delete asset and folder (Delete).....	7
4	Option B – OpenAPI-based AEM Assets APIs (Author + Folders)	8
4.1	When to use this option	8
4.2	Set up Adobe Developer Console (ADC) and OAuth S2S	8
4.3	Register the client in AEM via Cloud Manager	9
4.4	Verify service user permissions in AEM	9
4.5	Common variables & headers (OpenAPI)	10
5	Folder CRUD via Folders API (OpenAPI).....	10
5.1	List /content/dam/shrss/corporate/photography (Read)	10
5.2	Create a folder under photography (Create – folder)	11
5.3	Delete a folder (Delete – folder).....	11
6	Asset CRUD via Assets Author API (OpenAPI)	12
6.1	Get an asset’s ID from its path	12
6.2	Create a new asset via Import From URL (Create – asset)	12
6.3	Read asset metadata (Read).....	13
6.4	Update asset metadata (Update – metadata).....	13
6.5	Delete an asset (Delete)	14
7	Putting it together – recommended patterns for SHRSS	14

1 Overview

Classic Assets HTTP API vs OpenAPI-based Assets APIs

Audience: **SHRSS Technical Lead / Senior Architect** Scope: Using AEM APIs to perform folder and asset CRUD under:

- **Stage author:** <https://author-p135156-e1336256.adobe.com>
- **Base DAM folder:** /content/dam/shrss/corporate/photography

This guide compares and documents two approaches:

1. **Classic Assets HTTP API + Direct Binary Upload** (path-based /api/assets).
2. **OpenAPI-based AEM Assets Author API + Folders API** (ID-based + path-based).

The same CRUD flows are covered for each:

- List folder contents
- Create folder
- Create/import asset
- Update asset metadata (and binary where applicable)
- Delete asset and folder

2 Quick decision guide

Use this section to choose which approach to implement or maintain.

Dimension	Classic Assets HTTP API (/api/assets + Direct Binary Upload)	OpenAPI-based Assets APIs (Assets Author + Folders)
Primary use case	Existing path-based integrations, direct uploads from CI/CD, scripts tied to /content/dam/...	New greenfield integrations, modern, strongly-typed API surface
Path vs ID	Directly path-based (/api/assets/shrss/corporate/...)	Folders: path-based ; Assets Author: ID-based (urn:aaid:aem:...)
Auth mechanism	Technical account via AEM Developer Console + JWT access tokens	OAuth server-to-server via Adobe Developer Console (ADC)
Binary upload	Uses Direct Binary Upload (*.initiateUpload.json + signed URLs + *.completeUpload.json)	GA path = Import From URL ; direct binary upload still evolving (keep using classic where needed)
Folder CRUD	Via Assets HTTP API	Via Folders API (/adobe/folders/...)
Asset metadata CRUD	Via Assets HTTP API	Via Assets Author API (/adobe/assets/{assetId}/metadata)
Tooling & future-	Older, SIREN-based, still supported but not	OpenAPI (Swagger), SDK-friendly, modern

Dimension	Classic Assets HTTP API (/api/assets + Direct Binary Upload)	OpenAPI-based Assets APIs (Assets Author + Folders)
proofing	the long-term direction	error formats; long-term direction
Recommendation	Keep for existing, path-centric automation and direct binary upload	Preferred for all new integrations and for metadata/folder CRUD moving forward

Recommended strategy for SHRSS

- For **new** integrations and automation: **start with the OpenAPI-based APIs**.
- For existing workflows that:
 - rely on **path-based** logic, and/or
 - need **direct binary upload from pipelines** it is reasonable to **continue using the classic Assets HTTP API + Direct Binary Upload**, while planning a gradual transition.

3 Shared prerequisites (for both approaches)

Before using either API set, confirm the following for your SHRSS AEM program:

1. **Adobe IMS Org role**
 - You (or a designated admin) are an **IMS Org System Administrator** for the SHRSS org.
2. **AEM product profile**
 - That admin is a member of an AEM **Author** product profile with admin-level access, for example:
 - AEM Administrators - author - Program <ID> - Environment <ID>
 - This is required to:
 - Open **Cloud Manager** for the correct program.
 - Access the **AEM Developer Console** (classic path).
 - Configure API access via **Adobe Developer Console** (OpenAPI path).
3. **Cloud Manager access**
 - You can log into:
 - <https://experience.adobe.com/#/cloud-manager>
 - And see the SHRSS AEM program and environments (Dev/Stage/Prod).

If any of the above is missing, you will need your Adobe contact to help coordinate roles and access.

4 Option A – Classic Assets HTTP API + Direct Binary Upload

4.1 When to use this option

Use the classic Assets HTTP API when:

- You have **existing scripts/integrations** already using `/api/assets`.
- You need **direct binary upload** from CI/CD or on-prem sources and don't want to host temporary files externally.
- You want **path-based CRUD** aligned directly with `/content/dam/shrss/...`

The classic approach is documented as:

- Manage digital assets with the Adobe Experience Manager Assets HTTP API
- Developer references for Assets – Asset upload

4.2 Technical account & access token (classic)

At a high level:

1. In **Cloud Manager**, open the **Developer Console** for the target author environment.
2. Under **Tools → Integrations → Technical Accounts**, create a **technical account**.
3. Download the **service credentials JSON** (contains `client_id`, `technical_account_id`, etc.).
4. Implement the **JWT → access token** flow described here:
 - Generating Access Tokens for Server-Side APIs
 - How to obtain access tokens using AEM-CS API client code sample
5. Use the resulting **access token** in requests as:

```
Authorization: Bearer <ACCESS_TOKEN>
```

You can reuse the same technical account and token-generation code for both AEM author and `/api/assets` calls.

For examples below, assume:

```
export AEM_HOST="https://author-p135156-e1336256.adobecloud.com"
export AUTH="Authorization: Bearer <ACCESS_TOKEN>"
```

4.3 Folder & asset CRUD via Assets HTTP API

4.3.1 List folder contents (Read)

List contents of:

```
/content/dam/shrss/corporate/photography
curl -X GET \
  "$AEM_HOST/api/assets/shrss/corporate/photography.json" \
  -H "$AUTH"
```

Notes:

- Mapping is: /content/dam/shrss/corporate/photography → /api/assets/shrss/corporate/photography.json.
- Response is a **SIREN JSON** document with:
 - The folder's properties.
 - entities representing child folders and assets.

4.3.2 Create a folder under photography (Create – folder)

Example: create /content/dam/shrss/corporate/photography/api-demo

```
curl -X POST \
  "$AEM_HOST/api/assets/shrss/corporate/photography/api-demo" \
  -H "$AUTH" \
  -H "Content-Type: application/json" \
  -d '{
    "class": "assetFolder",
    "properties": {
      "title": "API Demo Folder"
    }
  }'
```

Verify:

```
curl -X GET \
  "$AEM_HOST/api/assets/shrss/corporate/photography/api-demo.json" \
  -H "$AUTH"
```

4.3.3 Create a new asset via Direct Binary Upload (Create – asset)

Target path:

```
/content/dam/shrss/corporate/photography/api-demo/sample.jpg
```

Step 1 – Initiate upload

```
FILE="./sample.jpg"
SIZE=$(stat -c%s "$FILE") # macOS: stat -f%z "$FILE"

curl -X POST \
  "$AEM_HOST/content/dam/shrss/corporate/photography/api-demo.initiateUpload.json" \
  -H "$AUTH" \
  -H "Content-Type: application/x-www-form-urlencoded; charset=UTF-8" \
  --data-urlencode "fileName=sample.jpg" \
  --data-urlencode "fileSize=$SIZE"
```

Response (simplified):

```
{
  "folderPath": "/content/dam/shrss/corporate/photography/api-demo",
  "files": [
    {
      "fileName": "sample.jpg",
      "mimeType": "image/jpeg",
      "uploadToken": "<UPLOAD_TOKEN>",
      "uploadURIs": ["<SIGNED_UPLOAD_URL>"],
      "minPartSize": 10485760,
      "maxPartSize": 104857600
    }
  ],
  "completeURI": "/content/dam/shrss/corporate/photography/api-demo.completeUpload.json"
}
```

Capture:

```
UPLOAD_URL="<SIGNED_UPLOAD_URL>"
UPLOAD_TOKEN="<UPLOAD_TOKEN>"
COMPLETE_URI="/content/dam/shrss/corporate/photography/api-demo.completeUpload.json"
```

Step 2 – Upload the binary

```
curl -X PUT \
  "$UPLOAD_URL" \
  -H "Content-Type: image/jpeg" \
  --data-binary @"$FILE"
```

Step 3 – Complete the upload (create the asset)

```
curl -X POST \
  "$AEM_HOST$COMPLETE_URI" \
  -H "Content-Type: application/x-www-form-urlencoded; charset=UTF-8" \
  --data-urlencode "fileName=sample.jpg" \
  --data-urlencode "mimeType=image/jpeg" \
  --data-urlencode "uploadToken=$UPLOAD_TOKEN"
```

Now you can retrieve the asset via:

```
curl -X GET \
  "$AEM_HOST/api/assets/shrss/corporate/photography/api-demo/sample.jpg.json" \
  -H "$AUTH"
```

Binary upload via /api/assets itself is deprecated; **Direct Binary Upload** is the recommended classic approach. See Developer references for Assets – Asset upload.

4.3.4 Update asset metadata & binary (Update)

Metadata update

```
curl -X PUT \
  "$AEM_HOST/api/assets/shrss/corporate/photography/api-demo/sample.jpg" \
  -H "$AUTH" \
  -H "Content-Type: application/json" \
  -d '{
    "class": "asset",
    "properties": {
      "dc:title": "Sample Image - Updated Title"
    }
  }'
```

Confirm:

```
curl -X GET \
  "$AEM_HOST/api/assets/shrss/corporate/photography/api-demo/sample.jpg.json" \
  -H "$AUTH"
```

Binary update (new version)

Repeat the **initiate** → **PUT to signed URL** → **complete** flow, but when calling completeUpload, pass versioning flags:

```
curl -X POST \
  "$AEM_HOST$COMPLETE_URI" \
  -H "Content-Type: application/x-www-form-urlencoded; charset=UTF-8" \
  --data-urlencode "fileName=sample.jpg" \
  --data-urlencode "mimeType=image/jpeg" \
  --data-urlencode "uploadToken=$UPLOAD_TOKEN" \
  --data-urlencode "createVersion=true" \
  --data-urlencode "versionLabel=v2" \
  --data-urlencode "versionComment=Reupload via API"
```

4.3.5 Delete asset and folder (Delete)

Delete the asset

```
curl -X DELETE \
  "$AEM_HOST/api/assets/shrss/corporate/photography/api-demo/sample.jpg" \
  -H "$AUTH"
```

Verify:

```
curl -X GET \
  "$AEM_HOST/api/assets/shrss/corporate/photography/api-demo/sample.jpg.json" \
  -H "$AUTH"
# Expect 404
```

Delete the folder

```
curl -X DELETE \
  "$AEM_HOST/api/assets/shrss/corporate/photography/api-demo" \
  -H "$AUTH"
```

List parent folder:

```
curl -X GET \
  "$AEM_HOST/api/assets/shrss/corporate/photography.json" \
  -H "$AUTH"
# api-demo should no longer appear in children
```

5 Option B – OpenAPI-based AEM Assets APIs (Author + Folders)

5.1 When to use this option

Use the OpenAPI-based APIs when:

- You are building **new workflows or services**.
- You want **stronger typing**, SDKs, and better tooling.
- You want to align with **Adobe's long-term direction** for AEM APIs.

Core documentation:

- AEM APIs overview
- Invoke OpenAPI-based AEM APIs for server to server authentication
- AEM Assets Author API
- AEM Folders API

5.2 Set up Adobe Developer Console (ADC) and OAuth S2S

In **Adobe Developer Console**:

1. Create an **AEM integration project** (or reuse an existing one) for SHRSS.
2. **Add APIs:**
 - **AEM Assets Author API**
 - **AEM Folders API** (if shown separately).
3. Configure **OAuth Server-to-Server** credentials.
4. Associate the integration with an appropriate **AEM product profile** that can reach the SHRSS environments.

Record:

- CLIENT_ID (API key)
- CLIENT_SECRET
- IMS_ORG
- Technical account principal
- OAuth S2S **scopes** (e.g. including `aem.assets.author`, `aem.folders`)

Token flow details: Invoke OpenAPI-based AEM APIs for server to server authentication

5.3 Register the client in AEM via Cloud Manager

In your Cloud Manager config repo, define allowed client IDs, for example in `api.yaml`:

```
kind: "API"
version: "1.0"
metadata:
  envTypes:
    - "dev"
    - "stage"
    - "prod"
data:
  allowedClientIDs:
    author:
      - "YOUR_CLIENT_ID_FROM_ADC"
```

Commit and run the **config pipeline** so that the SHRSS author environments accept calls from this client. See: AEM APIs overview.

5.4 Verify service user permissions in AEM

In **AEM Author (stage)**:

1. Go to **Tools → Security → Users**.
2. Find the user mapped to the ADC integration's technical account.

3. Ensure it has appropriate group membership (e.g. dam-users, dam-administrators, or a SHRSS-specific group) with read/write on:

```
/content/dam/shrss/corporate/photography
```

5.5 Common variables & headers (OpenAPI)

For the SHRSS **stage author** environment:

```
export AEM_HOST="https://author-p135156-e1336256.adobecloud.com"

# Host bucket (if used in examples)
export AEM_BUCKET="author-p135156-e1336256"

# OAuth S2S access token from ADC
export ACCESS_TOKEN="<ACCESS_TOKEN>"

# Client ID from ADC
export API_KEY="<CLIENT_ID>"

# Base folder
export BASE_FOLDER_PATH="/content/dam/shrss/corporate/photography"
```

Standard headers:

```
-H "Authorization: Bearer $ACCESS_TOKEN" \
-H "X-Api-Key: $API_KEY"
```

Many endpoints also accept:

```
-H "X-Adobe-Accept-Experimental: 1"
```

Token usage matches the patterns in Invoke OpenAPI-based AEM APIs for server to server authentication.

6 Folder CRUD via Folders API (OpenAPI)

Reference: AEM Folders API

6.1 List /content/dam/shrss/corporate/photography (Read)

```
curl -s -X GET \
  "$AEM_HOST/adobe/folders/?path=$BASE_FOLDER_PATH&limit=50" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "X-Api-Key: $API_KEY" \
  -H "Accept: application/json"
```

- path is the full **JCR path** under /content/dam.

- Response includes folder metadata and children.

6.2 Create a folder under photography (Create – folder)

Example: /content/dam/shrss/corporate/photography/api-demo-openapi

```
curl -s -X POST \
  "$AEM_HOST/adobe/folders/" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "X-Api-Key: $API_KEY" \
  -H "Content-Type: application/json" \
  -d '[
    {
      "path": "/content/dam/shrss/corporate/photography/api-demo-openapi",
      "title": "API Demo (OpenAPI)"
    }
  ]'
```

Notes:

- You can create multiple folders in one call by adding entries to the array.

6.3 Delete a folder (Delete – folder)

Delete /content/dam/shrss/corporate/photography/api-demo-openapi:

```
curl -s -X POST \
  "$AEM_HOST/adobe/folders/delete" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "X-Api-Key: $API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "paths": [
      "/content/dam/shrss/corporate/photography/api-demo-openapi"
    ],
    "recursive": true,
    "force": true
  }'
```

For async deletes (202 + jobId):

```
curl -s -X GET \
  "$AEM_HOST/adobe/folders/jobs/<JOB_ID>/result" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "X-Api-Key: $API_KEY"
```

7 Asset CRUD via Assets Author API (OpenAPI)

Reference: AEM Assets Author API

7.1 Get an asset's ID from its path

1. Call the asset's .json on author.
2. Extract jcr:uuid.
3. Build assetId as: urn:aaid:aem:<jcr:uuid>

Example:

```
ASSET_PATH="/content/dam/shrss/corporate/photography/example.jpg"
```

```
curl -s -X GET \
  "$AEM_HOST$ASSET_PATH.json" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "X-Api-Key: $API_KEY" \
  -H "Accept: application/json" \
  | jq -r '["jcr:uuid"]'
```

Suppose result:

```
94ac6f88-04a8-4937-872b-a0e971e6349c
```

Then:

```
export ASSET_ID="urn:aaid:aem:94ac6f88-04a8-4937-872b-a0e971e6349c"
```

You will use \$ASSET_ID with all Author API calls.

7.2 Create a new asset via Import From URL (Create – asset)

For /content/dam/shrss/corporate/photography/api-demo-openapi, you first obtain its UUID (TARGET_FOLDER_UUID) using the same .json + jcr:uuid pattern, then:

```
TARGET_FOLDER_UUID="<UUID_OF_/content/dam/shrss/corporate/photography/api-demo-openapi>"
TARGET_FOLDER_URN="urn:aaid:aem:$TARGET_FOLDER_UUID"
```

```
curl -s -X POST \
  "$AEM_HOST/adobe/assets/import/from-url" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "X-Api-Key: $API_KEY" \
  -H "X-Adobe-Accept-Experimental: 1" \
  -H "Content-Type: application/json" \
  -d '{
    "folder": \"$TARGET_FOLDER_URN",
    "files": [
```

```
{
  "fileName": "openapi-demo.jpg",
  "url": "https://example.org/path/to/openapi-demo.jpg"
}
```

Notes:

- url must be accessible (public or presigned).
- For **local-only** binaries, it's reasonable to continue using Direct Binary Upload (classic path) until all OpenAPI upload paths are GA.

7.3 Read asset metadata (Read)

```
curl -s -X GET \
  "$AEM_HOST/adobe/assets/$ASSET_ID/metadata" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "X-Api-Key: $API_KEY" \
  -H "X-Adobe-Accept-Experimental: 1" \
  -H "Accept: application/json"
```

You will see:

- assetId
- repositoryMetadata
- assetMetadata (customer metadata, e.g. dc:title, tags).

7.4 Update asset metadata (Update – metadata)

```
curl -s -X PATCH \
  "$AEM_HOST/adobe/assets/$ASSET_ID/metadata" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "X-Api-Key: $API_KEY" \
  -H "X-Adobe-Accept-Experimental: 1" \
  -H "Content-Type: application/json" \
  -d '{
    "assetMetadata": {
      "dc:title": "OpenAPI Demo - SHRSS Corporate Photography"
    }
  }'
```

- This uses patchAssetMetadata.
 - You send only fields you want to update.
-

7.5 Delete an asset (Delete)

```
curl -s -X DELETE \
  "$AEM_HOST/adobe/assets/$ASSET_ID" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "X-Api-Key: $API_KEY"
```

Responses may be synchronous (200) or async (202 + job). Check the OpenAPI spec for job polling if needed.

8 Putting it together – recommended patterns for SHRSS

For the SHRSS technical lead, a pragmatic plan:

1. Short term

- Keep existing **path-based** automations on the **Classic Assets HTTP API + Direct Binary Upload**.
- Use it for:
 - Direct uploads from CI/CD or on-prem sources.
 - Integrations heavily tied to /content/dam/shrss/... paths.

2. New integrations

- Use **OpenAPI-based Folders API** for folder CRUD.
- Use **OpenAPI-based Assets Author API** for metadata and import-from-URL.
- Implement auth via **ADC + OAuth S2S** once and reuse.

3. Migration path

- For each classic integration:
 - Move **metadata reads/updates** to **Assets Author API**.
 - Move **folder creation/deletion** to **Folders API**.
 - Keep Direct Binary Upload only where strictly needed for now.
- Over time, reduce dependencies on /api/assets to simplify support.

This approach lets you leverage the modern OpenAPI surface for most of your workflows today, while safely retaining the classic endpoints where they are still the best fit.